El Lenguaje



Orígenes

- *Desde un principio la programación fue pensada como una secuencia de procedimientos que actúan sobre datos.*
- *Los datos están totalmente separados de las funciones encargadas de su manipulación.*
- *Las funciones son consideradas cajas negras que producen cierta transformación en su salida.*

Orígenes

- *Dentro de este esquema se idearon métodos más eficientes, pero siempre con los datos separados.*
- *A pesar de haber mejorado mucho las herramientas, la complejidad creciente de los programas se transformaba en un obstáculo que crecía más rápido que las soluciones (Crisis del Software).*
- *Se imponía un cambio de modelo.*

Orígenes

- *La orientación a objetos modificó la forma de encarar el tema.*
- *Un lenguaje orientado a objetos permite tratar la complejidad de un programa, con bastante éxito.*
- *No solo es un enfoque distinto para la solución, sino que requiere reformular el problema en forma más natural.*

Técnicas de Programación

- *De acuerdo a la evolución histórica, podemos identificar las siguientes técnicas de programación, que también responden a la curva de aprendizaje de un individuo:*
  - *Programación NO estructurada*
  - *Programación procedural*
  - *Programación orientada a objetos*

Lenguajes Orientados a Objeto

- *Mejoran la productividad, sin imponer una estructura rígida.*
- *Permiten que un grupo de personas pueda coordinar las tareas de un gran proyecto con relativa facilidad.*
- *Permiten la reutilización del código.*
- *Mejoran el mantenimiento del código.*

C++

- *C++ es un lenguaje orientado a objetos.*
- *No es una simple extensión de C.*
- *Aunque se pueda hacer con C++ lo mismo que se hace con C, C++ es un concepto totalmente diferente.*
- *No por utilizar C++ se está programando con objetos.*

Conceptos fundamentales

- *Abstracción de datos*
  - *Significa combinar la estructura de datos y las operaciones sobre esa estructura en un nuevo tipo abstracto de datos.*
  - *Un tipo abstracto de datos no es parte de la definición del lenguaje, aunque se comporta como si lo fuera.*
  - *Los tipos de datos abstractos son creados por el programador, los denominamos clases.*

Conceptos fundamentales

- *Herencia*
  - *Es una de las herramientas más importantes para la reutilización de código.*
  - *A partir de una clase básica (ej. mamíferos), derivamos clases más específicas (ej. humanos).*
  - *La clase derivada agrega o elimina las particularidades que se necesitan de la clase base, para esto no es necesario tocar el código fuente.*

Conceptos fundamentales

- *Polimorfismo*
  - *Es la utilización de una interfaz común con diferentes implementaciones.*
  - *El polimorfismo mejora la calidad de un programa.*
  - *En C++ tenemos polimorfismo porque están totalmente separadas la interfaz de la implementación.*

Conceptos fundamentales

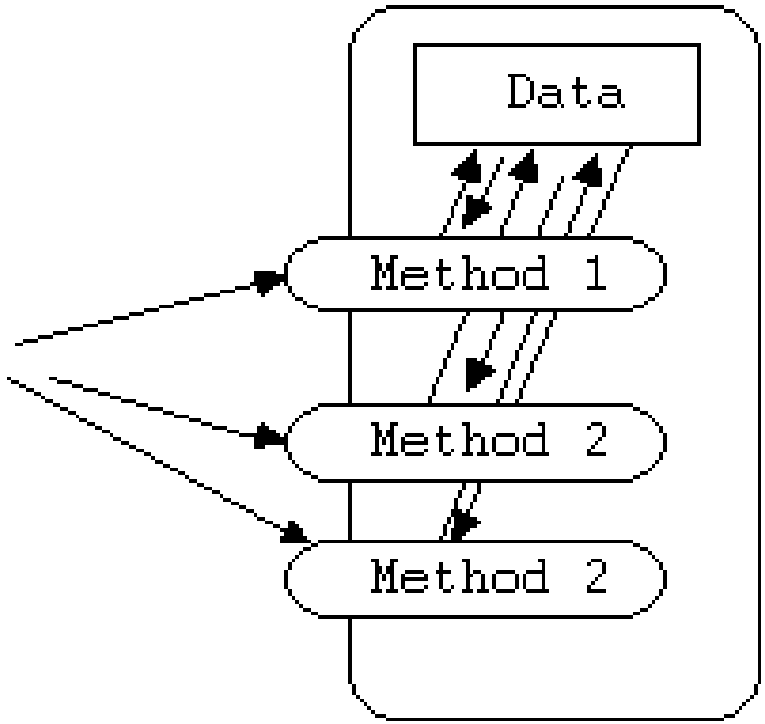
- *Polimorfismo (continuación)*
  - *El polimorfismo existe por lo que se denomina ligadura tardía (late binding) o dinámica.*
  - *Los lenguajes clásicos resuelven las direcciones de llamada a función en tiempo de compilación.*
  - *Los lenguajes polimórficos lo hacen en tiempo de ejecución.*

Definiciones

- *La actividad principal de la programación orientada a objetos es la generación de nuevos tipos de datos abstractos (clases).*
- *El usuario de una clase crea objetos y envía mensajes a los objetos.*
- *Un objeto debe saber cómo responder a un mensaje.*

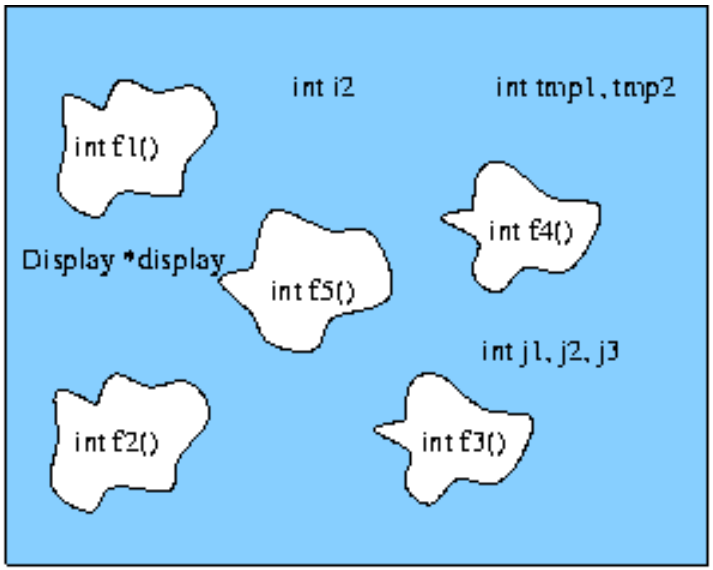
Objetos y Mensajes

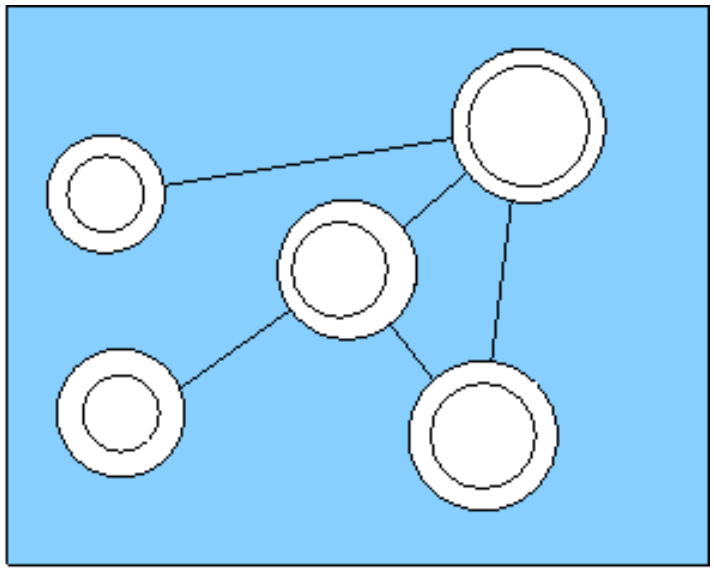
Se envía mensajes a un objeto, estos evocan los métodos que acce-den los datos internos (escondidos) del objeto.



Objetos y Mensajes

- *La programación orientada a objetos trata de cambiar la realidad de la figura de la izquierda en la de la derecha.*





Creación de Clases en C++

Veamos este programa

```
/*  Operaciones con números complejos en C */
#include <stdio.h>
#include <math.h>

typedef struct {
  double re, im;
} complejo;

complejo sumac( complejo a, complejo b);
complejo restac( complejo a, complejo b);
/* ... */
```

- *Si modificamos la estructura, ¿qué valor tienen las funciones?*

Consideraciones

- *Cualquier función es capaz de modificar los datos, no hay restricciones.*
- *Pensar en conjuntos de datos y en funciones que modifican esos datos es adaptar el problema al computador.*
- *¿Qué pasa si tratamos de hacer lo contrario? Adaptar el computador a una representación bastante más real de algo, digamos un objeto.*

Consideraciones

- *¿Qué perdemos con esto último? Nada.*
- *¿Qué ganamos? Mucho.*
- *Hemos visto que las funciones no tienen mucha utilidad si cambiamos la estructura de los datos que procesan y además que los datos están expuestos a que los modifique cualquiera.*
- *¿Qué pasa si agrupamos los datos con las funciones que los saben manejar?*

Consideraciones

- *Hemos creado una clase.*
- *La clase es algo abstracto, que se ve "instanciado" por un objeto.*
- *Esto significa que la clase es algo así como un nuevo tipo de datos y el objeto es una "variable" de esa clase.*
- *De esta forma incluimos en este nuevo tipo de datos, las funciones que saben "procesarlos".*

Veamos qué resulta

```
//  Operaciones con números complejos en C++
#include <iostream.h>
#include <math.h>
struct complejo {
    double re, im;
    void sumac( complejo a, complejo b);
    void restac( complejo a, complejo b);
    /* ... */
    void print();
};
```

- *Ahora tiene más sentido, los datos y las funciones que trabajan con ellos están declarados en el mismo lugar.*

Estructura ampliada

- *Está todo en un solo lugar y funciona de forma integrada.*
- *Si modificamos la estructura de datos, tenemos a la vista las funciones que intervienen y por lo tanto también modificamos su comportamiento.*
- *Para que esto tenga más sentido deberíamos asegurarnos que desde afuera no se puedan modificar los datos.*

Estructura ampliada

- *Veamos cómo se trabaja con esto:*

```
void main( void )
{
  complejo num1, num2, res;
  num1.re = 2.1 ; num1.im = 3.5;
  num2.re = 3.3 ; num2.im = 6.6;

  cout << "Compl 1: " ; num1.print() ; cout << "\n";
  cout << "Compl 2: " ; num2.print() ; cout << "\n";
  res.sumac( num1, num2 );
  cout << "La suma es: " ; res.print() ; cout << "\n";
  /* ... */
}
```

Estructura ampliada

- *Podemos inicializar y modificar los datos desde afuera.*
- *Esto no es exactamente lo que queremos ya que dijimos que solamente las funciones internas a la clase (o métodos) son idóneas para hacerlo.*
- *Es una forma de asegurar el tratamiento correcto de los datos.*

¿Cómo se resuelve?

```
struct complejo {
  private:
    double re, im;
  public:
    void init( double preal, double pimag );
    void sumac( complejo a, complejo b );
    void restac( complejo a, complejo b );
    /* ... */
    void print();
};
```

- *Ahora tenemos una parte privada que solo puede ser vista por las funciones (métodos) de la propia clase.*

Estructura ampliada

- *¿Porqué no colocamos los métodos en la parte privada?*
- *Porque no los podríamos utilizar si son inaccesibles fuera del objeto.*
- *Lo habitual es que los datos sean privados y los métodos sean públicos, aunque también solemos tener métodos privados que responden solamente a la convocatoria de otros métodos.*

Estructura ampliada

- *Por defecto (si no especificamos private), los componentes de una estructura son públicos.*
- *Podemos declarar*

```
private:
   ....
public:
   ....
```

  Tantas veces como querramos.

- *Nos acercamos a la definición de clase.*

Declaración de una clase

```
class complejo{
    double re, im;
  public:
    void init( double preal, double pimag );
    void sumac( complejo a, complejo b);
    void restac( complejo a, complejo b);
    /* ... */
    void print();
};
```

- *Como vemos, la declaración de una clase es muy parecida a la estructura ampliada.*

Declaración de una clase

- *Por defecto en una clase los elementos son privados.*
- *Por eso no escribimos*

  `private:`

  en la declaración anterior.
- *Aquí también podemos indicar varias veces miembros privados como públicos, alternando la declaración.*
- *Una clase nos permite definir objetos.*

Definición de objetos

`complejo` *ahora se comporta como un tipo*

```
void main( void )
{
  complejo num1, num2, res;
  num1.init( 2.1, 3.5 );
  num2.init( 3.3, 6.6 );
  /* ... */
}
```

- *La inicialización es realizada por una función miembro de la clase, no podemos hacerla con una función externa.*

Trabajo con objetos

- *Si queremos imprimir el contenido de un objeto, deberemos haber definido un método que lo haga y que se llama de la forma siguiente:*

```
num1.print();
```

- *Los paréntesis de apertura y cierre indican que es un método (que en este caso no lleva argumentos).*

Trabajo con objetos

- *Para referirnos a los miembros de los objetos, sean estos datos o funciones utilizamos la misma metodología que con las estructuras.*

Ejemplo:

```
complejo num1, *num2, &num3 = num1;
num1.print();
num2->print();
num3.print();
```

Constructores

- *La tarea de inicializar un objeto es total-mente rutinaria.*
- *Por esto se ha incluido como un proce-dimiento estándar en la definición de objetos.*
- *La inicialización la realiza un método denominado:* ***constructor***
- *Al constructor hay que declararlo y lleva el mismo nombre de la clase.*

Constructores

- *El constructor existe sin que lo defina-mos y se lo llama siempre.*
- *Si no lo definimos está vacío y su efec-to es nulo.*
- *Nunca se realiza una llamada explícita al constructor, es llamado por defecto cuando definimos un objeto.*
- *Al declarar el constructor, indicamos que realice lo que nosotros queremos.*

```
class complejo{
    double re, im;
  public:
    complejo( double preal, double pimag );
    void sumac( complejo a, complejo b);
    void restac( complejo a, complejo b);
    /* ... */
    void print();
};

int main( void ) {
  complejo num1( 2.1, 3.5 ), num2( 3.3, 6.6 ),
  res( 0.0, 0.0 );
  /* ... */
}

complejo::complejo( double preal, double pimag ) {
  re = preal;
  im = pimag;
}
```

Constructores

- *El constructor es una función que no puede devolver ningún tipo, es un error indicarle tipo devuelto.*
- *En el ejemplo siguiente vemos que cada vez que actúa el constructor aparte de inicializar el objeto, imprimirá un mensaje en la pantalla.*
- *La impresión del contenido de los objetos nos muestran su inicialización.*

```
class complejo{
    double re, im;
  public:
    complejo( double preal, double pimag );
    void sumac( complejo a, complejo b);
    void restac( complejo a, complejo b);
    /* ... */
    void print();
};

int main( void ) {
  complejo num1( 2.1, 3.5 ), num2( 3.3, 6.6 ),
  res( 0.0, 0.0 );
  cout << "num1: "; num1.print(); cout << endl;
  cout << "num2: "; num2.print(); cout << endl;
  /* ... */
}

complejo::complejo( double preal, double pimag ) {
  re = preal;
  im = pimag;
  cout << "Ejecutando el constructor" << endl;
}
```

Destructores

- *Así como el constructor es un método que actúa por defecto al definirse un objeto, también existe el destructor.*
- *Este actúa cuando el objeto desapare-ce.*
- *Una de sus mayores utilidades está en la recuperación de memoria para los objetos que utilizan memoria dinámica.*

Destructores

- *El destructor lleva el mismo nombre de la clase precedido de un tilde.*

```
class complejo{
  private:
    double re, im;
  public:
    complejo( double preal, double pimag );
    ~complejo();
    void sumac( complejo a, complejo b);
    /* ... */
    void print();
};
```

Destructores

- *Igual que con los constructores, los destructores no pueden tener un tipo devuelto.*
- *También existen y son llamados por defecto.*
- *Si no lo definimos explícitamente son vacíos y su efecto es nulo.*
- *Podemos ver cómo son llamados en el ejemplo siguiente.*

```
class complejo{
    double re, im;
  public:
    complejo( double preal, double pimag );
    ~complejo();
    void sumac( complejo a, complejo b);
    void restac( complejo a, complejo b);
    /* ... */
};

int main( void ) {
  complejo num1( 2.1, 3.5 ), num2( 3.3, 6.6 ),
  res( 0.0, 0.0 );
  /* ... */
}

complejo::complejo( double preal, double pimag ) {
  re = preal;
  im = pimag;
  cout << "Ejecutando el constructor" << endl;
}

complejo::~complejo() {
  cout << "Ejecutando el destructor" << endl;
}
```

Programa completo

- *A continuación vemos un programa completo que calcula e imprime la suma y resta de dos números complejos.*
- *Observar el constructor, el destructor y la forma en que es manejado el objeto resultado (res).*
- *El método signo() se declaró privado porque no es necesario ni deseable que se utilice desde el exterior.*

```
#include <iostream>  // P01-01.cpp
#include <math.h>

using namespace std;

class complejo {
    double re, im;
    char signo();
  public:
    complejo( double preal, double pimag );
    ~complejo();
    void sumac( complejo a, complejo b );
    void restac( complejo a, complejo b );
    void print();
};

int main( void ) {
  complejo num1( 2.1, 3.5 ), num2( 3.3, -0.6 ), res( 0.0, 0.0 );
  cout << "Primer complejo: " ; num1.print() ; cout << endl;
  cout << "Segundo complejo: " ; num2.print() ; cout << endl;
  res.sumac( num1, num2 );
  cout << "Suma : " ; res.print() ; cout << endl;
  res.restac( num1, num2 );
  cout << "Resta : " ; res.print() ; cout << endl;
}
```

```
complejo::complejo( double preal, double pimag ) {
  re = preal;
  im = pimag;
  cout << "Ejecutando el constructor (";
  cout << re << ", " << im << ")" << endl;
}

complejo::~complejo() {
  cout << "Ejecutando el destructor (";
  cout << re << ", " << im << ")" << endl;
}

void complejo::print() {
  cout << re << " " << signo() << "j " << fabs( im );
}

char complejo::signo() {
  if( im >= 0 )
    return( '+' );
  else
    return( '-' );
}
```

```
void complejo::sumac( complejo a, complejo b ) {
  re = a.re + b.re;
  im = a.im + b.im;
}

void complejo::restac( complejo a, complejo b ) {
  re = a.re - b.re;
  im = a.im - b.im;
}
```

La salida es:

sumac

restac

main

```
Ejecutando el constructor (2.1, 3.5)
Ejecutando el constructor (3.3, -0.6)
Ejecutando el constructor (0, 0)
Primer complejo: 2.1 +j 3.5
Segundo complejo: 3.3 -j 0.6
Ejecutando el destructor (2.1, 3.5)
Ejecutando el destructor (3.3, -0.6)
Suma: 5.4 +j 2.9
Ejecutando el destructor (2.1, 3.5)
Ejecutando el destructor (3.3, -0.6)
Resta: -1.2 +j 4.1
Ejecutando el destructor (-1.2, 4.1)
Ejecutando el destructor (3.3, -0.6)
Ejecutando el destructor (2.1, 3.5)
```

Actuación de los destructores

- *En la salida del programa anterior podemos ver que el constructor actúa al definir los miembros dato de un objeto, pero no con los parámetros formales de funciones.*
- *El destructor actúa cada vez que se elimina espacio de almacenamiento, en el caso de los métodos, el que se generó con el pasaje de argumentos por valor.*

¿Dónde se define un método?

- *Puede ser en la misma declaración de una clase:*

```
// Esto es un comentario de una línea
class complejo {
    double re, im;
     void signo() {
       if( im >= 0 )
         return( '+' );
       else
       return( '-' );
     }
  public:
    complejo( double preal, double pimag ) {
       re = preal; im = pimag;
     }
}
```

¿Dónde se define un método?

- *También se puede definir fuera de la declaración de la clase (como hicimos antes).*
- *Para esto se necesita del operador de resolución de ámbito "::"*

```
complejo::complejo( double preal, double pimag ) {
   re = preal;
   im = pimag;
 }
```

Operad. de Resolución de Ambito

- *Es posible definir una variable en una función miembro que tenga el mismo nombre que un miembro dato de la clase, se lo diferencia de la manera siguiente:*

```
void complejo::sumac( complejo a, complejo b ) {
  double re = 20;
  complejo::re = a.re + b.re;
  im = a.im + b.im;
  cout << "Versión local: " << re << endl; // imprime 20
  cout << "Versión de clase: " << complejo::re << endl;
}
```

Valores por Defecto

- *En C++ las funciones pueden otorgar a sus argumentos valores por defecto, esto es especialmente útil en los constructores.*
- *Podemos especificar todos los valores por defecto, o solo algunos, pero siempre comenzando desde la derecha, esto es, no podemos tener a la derecha de valores por defecto otros sin defecto.*

```
#include <iostream> // P01-02.cpp
#include <math.h>

using namespace std;

class complejo {
    double re, im;
    char signo();
  public:
    complejo( double preal = 0.0, double pimag = 0.0 );
    ~complejo() {}
    void print();
};

int main( void ) {
  complejo num1( 2.1, 3.5 ), num2( 3.3 ), res;
  cout << "Primer complejo: " ; num1.print() ; cout << endl;
  cout << "Segundo complejo: " ; num2.print() ; cout << endl;
  cout << "Resultado: " ; res.print() ; cout << endl;
}

complejo::complejo( double preal, double pimag ) {
  re = preal;
  im = pimag;
}
```

```
Salida:
Primer complejo: 2.1 + j3.5
Segundo complejo: 3.3 + j0
Resultado: 0 + j0
```

Variaciones en la notación

- *Bajo la denominación: Inicialización de Objetos, podemos observar variantes en la inicialización de miembros dato en los constructores.*
- *Esto parte de la premisa de que en C++ todos son objetos, incluyendo las variables de tipos básicos.*
- *Esta notación simplifica la inicialización de miembros dato de los objetos.*

```
#include <iostream>  // P01-03.cpp
#include <math.h>

using namespace std;

class complejo {
    double re, im;
    char signo();
  public:
    complejo( double preal = 0.0, double pimag = 0.0 );
    ~complejo() {}
    void print();
};

int main( void ) {
  complejo num1( 2.1, 3.5 ), num2( 3.3 ), res;
  cout << "Primer complejo: " ; num1.print() ; cout << endl;
  cout << "Segundo complejo: " ; num2.print() ; cout << endl;
  cout << "Resultado: " ; res.print() ; cout << endl;
}

complejo::complejo(double preal, double pimag) : re(preal),
im(pimag) {}  // Todo esto puede ir en una sola línea.
```

```
Salida:
Primer complejo: 2.1 + j3.5
Segundo complejo: 3.3 + j0
Resultado: 0 + j0
```

Constructor de Copia

- *El constructor de copia crea un objeto a partir de otro objeto existente.*
- *En este caso se tiene solo un argumento que es una referencia al objeto desde el cual se está copiando.*
- *La forma general es la siguiente:*

```
tipo_clase::tipo_clase(const tipo_clase &obj);
```

```
#include <iostream.h>  // P01-04.cpp
#include <math.h>

using namespace std;

class complejo{
    double re, im;
  public:
    // Constructor
    complejo( double preal=0.0 , double pimag=0.0 ) :
                                re(preal), im(pimag) {};
    // Constructor de copia
    complejo( const complejo &cp ) : re(cp.re), im(cp.im) {}
    void print();
    char signo();
};

int main( void ) {
  complejo num1( 2.1, 3.5 );
  complejo num2( num1 );
  cout << "num1: "; num1.print(); cout << endl;
  cout << "num2: "; num2.print(); cout << endl;
}
```

```
void complejo::print() {
  cout << re << " " << signo() << "j " << fabs( im );
}

char complejo::signo() { return im >= 0 ? '+' : '-'; }
```

```
Salida:
num1: 2.1 +j 3.5
num2: 2.1 +j 3.5
```